

Architecture questions for the CTO

Questions to settle the architecture behind the second-demo requirements, each with why it matters and what it unblocks.

How to use this

- **Short on time?** Ask the eight priority questions below. They settle decisions that block the foundations.
- **Full list:** 29 questions in eight areas (A–H). Each says why it matters and which part of the plan it unblocks.
- **After the call:** record each answer as a decision in docs/DECISIONS.md and update the approach plan.

If you only have 15 minutes

#	Question	Why it can't wait
A3	Database: PostgreSQL on OCI (managed service or self-managed), or does Fusion standardise on Oracle Database / Autonomous DB?	The platform relies on PostgreSQL features: row-level security for tenancy, and exclusion constraints for effective dating. Moving to Oracle DB would be a major rewrite, so this must be settled before building the foundations.
B1	On the call the client heard “small microservices”. Today the platform is a modular monolith with strict module boundaries (decision D-02). Do we describe it as that, or is there a plan to split services out?	Ashley will read the architecture document against what was said. It must be accurate: a well-bounded modular monolith is a defensible answer; claiming microservices we don't have is not.
A1	The client was told each customer gets its own install on a common code base. Is that the rule for every customer, or can smaller customers share one install?	We built shared-database multi-tenancy with row-level security. One tenant per VM works with no code change, but the answer decides how many installs we operate, how upgrades roll out and what we promise in the architecture document.
C1	Effective dating: is our proposal the right depth — “valid from / valid to” on every changing fact, plus when the change was recorded — or does Fusion already have an HR-style pattern to reuse (e.g. from Oracle HCM work)?	The client explicitly compared it to an HR system. Reusing a proven Fusion pattern lowers risk; this is the biggest foundation in the plan and hard to change later.
D2	System of record: between the student platform and GrantsNow, which system owns person, student, award and funding data?	“Single source of truth” only works if each fact has one owner. Otherwise both systems write the same data and drift apart.
D3	Identity: should users sign in through institutional SSO (Microsoft Entra ID, SAML / Shibboleth) or a shared Fusion identity with GrantsNow?	Today the platform has its own login (JWT); there is no SSO. Universities will expect SSO, and it affects roles and permissions across products.
F2	AI features in the product currently call a third-party model provider (Groq / OpenRouter). Is that acceptable for student data, or must models run inside OCI (e.g. OCI Generative AI) or be switched off by default?	Student data is personal data. Where prompts are processed is a data-protection question, and it decides how far the AI layer (e.g. custom-attribute governance) can go.
H1	Do you agree with foundations first (versioning, effective dating), shown through thin end-to-end slices, even though the client ranked XML and API as “essential”?	The client's Tier 1 asks depend on the foundations. Building Tier 1 on today's model would mean rebuilding it. This needs a CTO-level decision and message to the client.

Where we stand today (for reference on the call)

Area	Current state
Architecture	Modular monolith (decision D-02): FastAPI / Python backend, Next.js 14 frontend, PostgreSQL
Tenancy	One database and schema; tenant_id on ~95 tables; PostgreSQL row-level security; tenant from subdomain + login token; fail-closed application role
Deployment	Documented for one or two generic VMs; no Oracle Cloud material yet
Sign-in	Platform's own login (JWT); no institutional SSO
Integration	Integration hub with finance / HR / research adapters, outbox, signed webhooks, OpenAPI docs; no third-party API keys; no scheduled publication
Statutory	One HESA student pack, flat student-level record, CSV output; mapping, advisory ingest, sign-off
Effective dating	Lifecycle events, supervisors and funding dated; status, modules and UOA not
AI in the product	Third-party model providers (Groq / OpenRouter); every AI feature has a non-AI fallback
Quality	575 automated tests; database migrations; decision records

A. Deployment, tenancy and hosting

#	Question	Why it matters	Unblocks
A1	The client was told each customer gets its own install on a common code base. Is that the rule for every customer, or can smaller customers share one install?	We built shared-database multi-tenancy with row-level security. One tenant per VM works with no code change, but the answer decides how many installs we operate, how upgrades roll out and what we promise in the architecture document.	Deployment model, release process
A2	Hosting on the same Oracle Cloud tenancy as GrantsNow: which region, and how are customers isolated from each other (separate compartments, VCNs, databases)?	UK universities will ask where their data lives and how it's separated from other customers' data. Their security review will test this before signing.	OCI deployment guide, security review
A3	Database: PostgreSQL on OCI (managed service or self-managed), or does Fusion standardise on Oracle Database / Autonomous DB?	The platform relies on PostgreSQL features: row-level security for tenancy, and exclusion constraints for effective dating. Moving to Oracle DB would be a major rewrite, so this must be settled before building the foundations.	Effective dating, tenancy, hosting
A4	What does “unified disaster recovery” with GrantsNow mean in targets — RPO (how much data can be lost) and RTO (how long to restore)?	Backup frequency, standby design and cost all follow from these numbers. We currently document backups and single-tenant restore, not DR targets.	DR design, SLA
A5	How are releases rolled out to many customer installs — CI/CD pipeline, maintenance windows, who runs them, can a customer defer an upgrade?	With federated installs, every release is N deployments. Regulatory (HESA) updates have fixed deadlines, so a customer who defers upgrades could miss a return.	Release process, regulatory updates

B. Architecture style

#	Question	Why it matters	Unblocks
B1	On the call the client heard “small microservices”. Today the platform is a modular monolith with strict module boundaries (decision D-02). Do we describe it as that, or is there a plan to split services out?	Ashley will read the architecture document against what was said. It must be accurate: a well-bounded modular monolith is a defensible answer; claiming microservices we don't have is not.	Architecture document, client trust
B2	If anything is split out, what first — the statutory/return engine, the integration and publication layer, or the AI features?	Heavy work (building returns, data publication) is what could slow the app for users. Knowing what may be split lets us keep those modules' boundaries clean now.	Integration, statutory engine
B3	Is there a Fusion-wide architecture standard we should follow (reference architecture, decision records, review board)?	The platform should fit Fusion's other products. We keep decision records in docs/DECISIONS.md and can align with an existing process.	Governance

C. Data model foundations

#	Question	Why it matters	Unblocks
C1	Effective dating: is our proposal the right depth — “valid from / valid to” on every changing fact, plus when the change was recorded — or does Fusion already have an HR-style pattern to reuse (e.g. from Oracle HCM work)?	The client explicitly compared it to an HR system. Reusing a proven Fusion pattern lowers risk; this is the biggest foundation in the plan and hard to change later.	Effective dating (WS3)
C2	Are we allowed to depend on PostgreSQL-specific features, or must the model stay portable to another database?	Our design uses PostgreSQL exclusion constraints to guarantee no overlapping dates. Portability would mean enforcing that only in application code.	Effective dating, versioning
C3	Programme / module versioning for the CMA rule: should versions be immutable once students are enrolled, and who approves a new version?	Immutability is the simplest way to guarantee a cohort stays on what it signed up for. It affects the data model, the UI and the approval workflow.	Catalogue & versioning (WS1)
C4	Custom attributes: what performance limits should we design to — maximum attributes per customer, and page / report response times?	The client's other product grew to ~1000 custom fields. Explicit limits turn “make it fast” into testable targets and justify the approval gate and soft cap.	Custom attribute governance (WS7)
C5	GDPR: how do we reconcile “never delete history” (effective dating, audit) with retention limits and the right to erasure?	Effective dating keeps every past value by design. We need an agreed retention and anonymisation policy before history accumulates, not after.	Effective dating, compliance

D. Integration and data

#	Question	Why it matters	Unblocks
D1	Is there a Fusion integration standard or platform we must use (e.g. Oracle Integration Cloud, a common API gateway, an event bus)?	The client wants an object-based API and scheduled warehouse publication. If GrantsNow already uses a standard, we should plug into it rather than build a separate one.	Integration API (WS6)
D2	System of record: between the student platform and GrantsNow, which system owns person, student, award and funding data?	“Single source of truth” only works if each fact has one owner. Otherwise both systems write the same data and drift apart.	GrantsNow integration, finance demo
D3	Identity: should users sign in through institutional SSO (Microsoft Entra ID, SAML / Shibboleth) or a shared Fusion identity with GrantsNow?	Today the platform has its own login (JWT); there is no SSO. Universities will expect SSO, and it affects roles and permissions across products.	Security, integration
D4	Data warehouse publication: which target (Oracle ADW / Analytics, or whatever the customer runs), push or pull, which formats?	Direct database access was declined on the call; publication is the alternative we offered. The target decides the connector we build first.	Data publication (WS6)
D5	Can larger installs have a read replica for reporting and extracts?	Publishing and heavy reports from a replica protects the live app — the reason given for declining direct database access.	Data publication, performance

E. Statutory and regulatory

#	Question	Why it matters	Unblocks
E1	Who owns the yearly HESA specification updates, and what turnaround do we commit to under the subscription?	Regulatory updates were promised under licence. Someone must track HESA releases and ship spec packs before each collection window.	Regulatory updates, contract
E2	Does Fusion (or any customer) have experience of HESA Data Platform submission — is there an API we can use, or is upload the only route?	“Publish directly to HESA” was described as the ideal end state. If no API exists we should say so early and keep download / upload.	Direct submission (WS5)
E3	Beyond HESA Data Futures, which statutory bodies are in scope (Scottish funding council, UKVI reporting, others)?	The spec-pack model should cover all of them. The client flagged England / Scotland differences.	Statutory engine (WS4)

F. Security and compliance

#	Question	Why it matters	Unblocks
F1	What will the client's security review expect — penetration test, ISO 27001 / Cyber Essentials, encryption at rest, audit-log retention?	These are usually conditions of signing. Knowing them now lets us build evidence as we go instead of retrofitting.	Security evidence
F2	AI features in the product currently call a third-party model provider (Groq / OpenRouter). Is that acceptable for student data, or must models run inside OCI (e.g. OCI Generative AI) or be switched off by default?	Student data is personal data. Where prompts are processed is a data-protection question, and it decides how far the AI layer (e.g. custom-attribute governance) can go.	AI features, compliance

G. Engineering quality (the AI-code concern)

#	Question	Why it matters	Unblocks
G1	What evidence of engineering quality does the CTO want to show the client — test coverage, static analysis, code review, architecture reviews?	Ashley raised brittle AI-generated code as a concern and the answer was accepted as a watch item. Concrete evidence (575 automated tests today) is more convincing than assurances.	Architecture & QA document (WS8)
G2	Who reviews AI-assisted code before it merges, and should we record that process in the architecture document?	A named human review step is the direct answer to “who is overseeing what the AI produces”.	Client trust

H. Delivery and priorities

#	Question	Why it matters	Unblocks
H1	Do you agree with foundations first (versioning, effective dating), shown through thin end-to-end slices, even though the client ranked XML and API as “essential”?	The client's Tier 1 asks depend on the foundations. Building Tier 1 on today's model would mean rebuilding it. This needs a CTO-level decision and message to the client.	Roadmap
H2	Next demo: the finance payments flow (agreed in principle) or visible Tier 1 progress?	Both compete for the same time. The payments flow also needs GrantsNow endpoints and a test environment.	Next demo
H3	“Features released disabled, enabled per customer”: plain per-install settings, or central licence management that controls what each customer is entitled to?	A per-install toggle is simple; licence-controlled entitlement needs a central service. The call mentioned licence management.	Feature flags (WS9)
H4	What scale should we design and test for — students per customer, concurrent users, return build time, uptime?	Non-functional targets turn “it must be fast” into numbers for load tests and hosting size.	Hosting, performance

Red-marked question IDs are in the 15-minute list.

After the call

- Record each answer as a decision (next numbers after D-04) in docs/DECISIONS.md.
- Update the Demo 2 approach plan where an answer changes a workstream, especially A3 (database), B1 (architecture style) and H1 (sequencing).
- Feed confirmed answers into the architecture document the client asked for.